

Компютърни мрежи и протоколи – OSI модел. Протоколи IPv4, IPv6, TCP, HTTP

Ивайло Андреев + gemini 3 Flash

31 май 2026

Съдържание

1	OSI модел – най-обща характеристика на нивата, съпоставяне с модела TCP/IP	2
1.1	OSI модел - най-обща характеристика на нивата	2
1.2	Съпоставяне между моделите OSI и TCP/IP	4
2	Протокол IPv4 адресация – класова и безкласова	5
2.1	Структура на адреса	5
2.2	Класова адресация (Classful Addressing)	6
2.3	Безкласова адресация (Classless Addressing - CIDR)	7
3	Основни характеристики на протокол IPv6	7
3.1	Ключови характеристики	7
3.2	Типове IPv6 адреси	8
4	TCP – процедура на трикратно договаряне	9
5	Хипертекстов протокол HTTP	9
5.1	Основни принципи и транспорт	10
5.2	Структура на съобщенията и методи	10

1 OSI модел – най-обща характеристика на нивата, съпоставяне с модела TCP/IP

1.1 OSI модел - най-обща характеристика на нивата

OSI (Open Systems Interconnection) моделът е теоретичен абстрактен модел, описващ принципите на комуникация в мрежа и устройството на мрежовите протоколи. Той е въведен от организацията **ISO**. Основната градивна единица са словете (нивата), като всеки слой предоставя интерфейс и услуги към слоя над него. Моделът се състои от следните 7 нива, номерирани отдолу нагоре:

1. **Физически слой (Physical Layer):** Най-ниският слой на OSI модела. Отговаря за приемането и предаването на неструктурирани потоци от данни (битове) по физическия носител. Той осигурява физическата връзка между върховете в дадена топология. Неговите функции включват:
 - *Синхронизация на битовете* – осигурява механизми за синхронизация между изпращача и получателя.
 - *Честота на изпращане на битове (bit rate control)*.
 - *Определяне на мрежовата топология* (bus, star, mesh).
 - *Режим на предаване на битовете* – simplex, half-duplex или full-duplex.
 - *Начин на установяване и прекъсване на връзката между устройства*.
 - *Кодиране и декодиране на информацията* (в двоичен вид) към и от електрически или физически сигнали.Обектите на този слой са хардуерни устройства: мрежови карти (NIC), модеми, повторители (hubs) и коаксиални кабели.
2. **Канален слой (Data Link Layer - DLL):** Грижи се за безгрешното и надеждно предаване на данни от едно устройство до друго, утилизирайки физическия слой. Разделя се на два подслоя: **Logical Link Control (LLC)** и **Media Access Control (MAC)**. Той предава пакетите до правилните хостове, използвайки MAC адресите на устройствата, като за преобразуване между IP и MAC адреси често се използва **ARP (Address Resolution Protocol)**. Когато пакет от мрежовия слой пристигне, той бива разбиван на кадри (frames) спрямо размера, дефиниран от мрежовата карта. Функциите му са:
 - *Разбиване на кадри (framing)* – организира последователност чрез поставяне на специални битови последователности в началото и края на всеки кадър.
 - *Физическо адресиране* – добавяне на MAC адресите на изпращача/получателя в хедъра на кадъра.

- *Контрол над грешките* – откриване и повторно предаване на счупени или загубени кадри.
- *Контрол над потока (flow control)* – координация на количеството данни, предавани за единица време, поради различната скорост на устройствата.
- *Контрол на достъпа* – MAC подслоят определя кой има контрол над споделения комуникационен канал в даден момент.

Имплементира се от Ethernet, суичове (switches) и бриджове (bridges).

3. **Мрежови слой (Network Layer):** Отговаря за предаването на данни между хостове от различни мрежи, които могат да се различават по своите физически и канални слоеве. Данните се капсулират под формата на пакети (**PDU – Protocol Data Unit**). Основни функции:

- *Маршрутизация (Routing)* – намиране на най-кратък/оптимален път от изпращача до получателя през мрежата.
- *Логическа адресация* – уникална идентификация на всяко устройство в глобалната мрежа чрез поставяне на IP адреси на изпращача и получателя в хедъра на пакета.
- *Предотвратяване на натоварването (congestion control)* в мрежата.

Примерни протоколи: **IP (IPv4/IPv6)**, **ICMP**, **IPSec**.

4. **Транспортен слой (Transport Layer):** Осигурява доставка на съобщенията от край-до-край (end-to-end). Протоколите от този слой разбиват съобщенията, получени от сесийния слой, на по-малки единици, наречени **сегменти**, и ги подават на мрежовия слой. При получателя се грижи за реасемблирането им в правилната наредба и цялост. Функции:

- *Сегментация и повторно сглобяване* – добавяне на метаданни в хедъра за реда на реасемблиране на сегментите.
- *Адресация на услуги (Service Point Addressing)* – добавяне на порт на източника и дестинацията в хедъра на всеки сегмент, за да се достави съобщението до правилния процес.

Този слой се имплементира като част от операционната система. Примери: **TCP** (ориентиран към връзка, надежден) и **UDP** (без връзка, лек и бърз).

5. **Сесиен слой (Session Layer):** Отговаря за управлението на диалога между приложенията. Предоставя:

- *Установяване, поддръжка и терминиране на сесии* между комуникиращи програми.
- *Синхронизация* – добавяне на контролни точки (checkpoints) в потока от данни, което позволява възстановяване от мястото на прекъсване без загуба на информация.

- *Диалогов контролер* – следи чий ред е да изпраща данни, поддържайки режими simplex, half-duplex или full-duplex.
- *Сигурност* – извършване на процеси по автентикация.

Обичайно се имплементира чрез сокети (sockets).

6. **Презентационен слой (Presentation Layer):** Извлича и манипулира данните от приложния слой, грижейки се за синтаксиса и формата им, за да станат годни за пренос. Предоставя:

- *Транслация* – конвертиране между различни формати за представяне на данни.
- *Криптиране и декриптиране* – защита на информацията.
- *Компресия* – намаляване на обема на пренасяните данни.

Примери: **TLS, SSL, MIME.**

7. **Приложен слой (Application Layer):** Най-горният слой, съвкупност от протоколи, които се имплементират директно от потребителските и сървърните процеси и приложения. Позволява на потребителя достъп до мрежови услуги. Примери: **HTTP, DNS, FTP, SMTP.**

1.2 Съпоставяне между моделите OSI и TCP/IP

Докато OSI е теоретичен наръчник, описващ абстрактно функциите, моделът **TCP/IP** е разработен на базата на реално съществуващи стандартни протоколи. TCP/IP е по-компактна версия и се състои от 4 слоя:

1. **Мрежов интерфейс (Network Interface / Достъп до мрежата):** Обединява функциите на *Физическия* и *Каналния* слой на OSI.
2. **Мрежов слой (Internet / Маршрутизация):** Директно съответства на *Мрежовия* слой на OSI.
3. **Транспортен слой (Transport / Транспортиране):** Съответства на *Транспортния* слой на OSI.
4. **Приложен слой (Application / Приложение):** Обхваща и събира функциите на трите горни слоя на OSI – *Сесиен, Презентационен* и *Приложен*.

Общи свойства:

- И двата модела се базират на единен стек от независими протоколи.
- Слоевете изпълняват сходни базови функции по обработка и транспорт на данните.

Разлики:

- *Разграничение на концепциите:* OSI прави ясно и строго разграничение между три основни свойства – услуги (какво прави слоят), интерфейси (как се достъпва) и протоколи (как се имплементира). При TCP/IP няма такова ясно разграничение.
- *Време на създаване:* OSI моделът е измислен преди протоколите за него, което го прави по-абстрактен и универсален. Моделът TCP/IP е създаден след протоколите, нагоден е към тях и подмяната им е по-трудна.
- *Връзка на транспортно ниво:* В класическия OSI модел мрежовото ниво поддържа connection-oriented режим, докато при TCP/IP интернет слой е изцяло connectionless (IP), а транспортният слой предоставя избор между connectionless (UDP) и connection-oriented (TCP) режими.

2 Протокол IPv4 адресация – класова и безкласова

IPv4 е фундаментален протокол от мрежовия слой, чиято основна задача е успешното предаване на пакети от източник до получател в рамките на глобалната мрежа. Това се постига чрез присвояване на уникален IP адрес на всеки хост и маршрутизатор.

2.1 Структура на адреса

Адресите в IPv4 са 32-битови двоични числа, които стандартно се записват като 4 десетични числа (октета), разделени с точка (напр. 192.0.2.1), като всеки октет е в интервала [0, 255]. Всеки адрес логически се състои от две части:

- **Мрежов идентификатор (netid / префикс)** – идентифицира конкретната мрежа.
- **Идентификатор на хоста (hostid)** – идентифицира конкретното устройство в тази мрежа.

Специални адреси в хост частта:

- Ако хост частта съдържа само двоични нули (**0**), това е *адресът на самата мрежа*.
- Ако хост частта съдържа само единици (**1**), това е *broadcast адресът* за едновременно предаване до всички хостове в тази мрежа.
- **0.0.0.0** – обозначава неопределен адрес (unspecified address); записът **0.0.0.0/0** се използва като маршрут по подразбиране (default route) в таблиците за маршрутизация.

2.2 Класова адресация (Classful Addressing)

В исторически план IPv4 адресите са били разделени на 5 фиксирани класа (А, В, С, D, Е), характеризиращи се с твърд префикс и мрежова маска по подразбиране. Максималният брой използвани хостове в дадена подмрежа се изчислява по следната математическа формула:

$$N_{\text{hosts}} = 2^n - 2 \quad (1)$$

където n е броят на битовете, заделени за хостовата част на адреса.

- **Клас А:**

- Фиксиран първи бит: **0**.
- Обхват на първия октет: **1.0.0.0** до **126.255.255.255** (Адрес 127.0.0.1 е заделен за loopback тест).
- Маска по подразбиране: **255.0.0.0** (/8 префикс).
- Капацитет: 127 мрежи с по над 16 милиона хоста всяка ($2^{24} - 2$).

- **Клас В:**

- Фиксирани първи битове: **10**.
- Обхват на първия октет: **128.0.0.0** до **191.255.255.255**.
- Маска по подразбиране: **255.255.0.0** (/16 префикс).
- Капацитет: около 16,000 мрежи с по 65,534 хоста всяка ($2^{16} - 2$).

- **Клас С:**

- Фиксирани първи битове: **110**.
- Обхват на първия октет: **192.0.0.0** до **223.255.255.255**.
- Маска по подразбиране: **255.255.255.0** (/24 префикс).
- Капацитет: над 2 милиона мрежи (2^{21}), но само с по 254 хоста всяка ($2^8 - 2$).

- **Клас D:**

- Фиксирани първи битове: **1110**.
- Обхват: **224.0.0.0** до **239.255.255.255**.
- Предназначение: Резервиран изцяло за **multicasting** (групово предаване). Няма subnet маска и хост адреси, тъй като не адресира единичен хост.

- **Клас Е:**

- Фиксирани първи битове: **1111**.
- Обхват: **240.0.0.0** до **255.255.255.254**.

- Предназначение: Резервиран за експериментални цели и бъдещи изследвания. Няма subnet маска.

Недостатъци: Класовата система се оказва изключително негъвкава. Половината от адресното пространство се заема от клас А за едва 127 системи. С разрастването на Интернет адресите от клас В се изчерпват светкавично, което принуждава раздаването на множество класове С на големи организации. Това води до драстично раздуване на глобалните маршрутизиращи (рутиращи) таблици, тъй като маршрутизаторите трябва да пазят записи за всяка отделна малка мрежа без възможност за агрегиране.

2.3 Безкласова адресация (Classless Addressing - CIDR)

За решаване на проблема с изчерпването на адресите и раздуването на таблиците се въвеждат техниките **VLSM (Variable-Length Subnet Masking)** и безкласовата адресация **CIDR (Classless Inter-Domain Routing)**.

При CIDR твърдите граници на класовете отпадат. Всеки адрес се представя като 32-битово число, последвано от наклонена черта „/“ и число, указващо точния брой на единиците в мрежовата маска (дължина на префикса). Например, в CIDR запис **192.0.2.96/28**, префиксът за мрежа (*netid*) заема първите 28 бита, а за хостове остават $32 - 28 = 4$ бита ($2^4 - 2 = 14$ използвани хоста). Съответната подмрежова маска е **255.255.255.240**. Това позволява мрежите да се разпределят с гъвкав размер и да се агрегират (супернетинг) с цел оптимизация на рутиращите таблици.

3 Основни характеристики на протокол IPv6

Поради окончателното изчерпване на IPv4 адресите, преходът към **IPv6** става наложителен, макар и бавен поради липсата на директна обратна съвместимост. IPv6 е проектиран да реши фундаменталните проблеми на своя предшественик.

3.1 Ключови характеристики

- **Огромно адресно пространство:** Адресите в IPv6 са **128-битови**. Те предоставят практически неизчерпаем брой уникални адреси.
- **Опростен и разширяем хедър:** Много от тромавите и рядко използвани полета от IPv4 хедъра са премахнати или изнесени като опционални разширения (Extension Headers) в края на хедъра. Така базовият хедър остава с фиксиран и изчистен формат, което позволява двойно по-бърза обработка от рутерите, въпреки че самите адреси са 4 пъти по-дълги.
- **Комуникация от край-до-край (End-to-End):** Всеки хост притежава глобално уникален публичен IP адрес. На практика това премахва необходимостта от NAT в повечето сценарии, възстановявайки оригиналния модел на

директна комуникация, въпреки че в практиката все още съществуват форми на трансляция като NAT66.

- **Автоматична конфигурация:** Поддържат се както *stateful* (чрез DHCPv6), така и *stateless* (SLAAC – **Stateless Address Autoconfiguration**) автоматична конфигурация на устройствата. Устройствата могат сами да генерират своя IP адрес без наличие на DHCP сървър.
- **Интегрирана сигурност:** Протоколът IPv6 е проектиран с вградена поддръжка на **IPSec**, осигуряващ рамка за автентикация и криптиране, въпреки че реалното му прилагане и използване не е задължително във всички съвременни софтуерни реализации.
- **Липса на Broadcast:** В IPv6 напълно отсъстват broadcast адресите. Тяхната функция се поема изцяло от по-ефективното **multicast** (групово) адресиране, предотвратявайки излишното натоварване на всички хостове в сегмента.
- **Anycast адресиране:** Един и същ IPv6 адрес може да бъде присвоен на група хостове (напр. разпръснати сървъри), като пакетите автоматично се рутират до най-близкия от тях.
- **Мобилност:** Позволява на хостовете физически и географски да сменят мрежата си, като на практика запазват текущия си IP адрес благодарение на автоконфигурацията и специфичните разширения в хедърите.

3.2 Типове IPv6 адреси

Адресите се записват като осем групи от по четири шестнадесетични цифри, разделени с двоеточие. За избягване на преливане извън текстовото поле при по-дълги адреси се използва безопасен режим на визуализация, например:

2001:0db8:0000:0000:0000:ff00:0042:8329.

Основните дефинирани типове са:

- **Неопределен адрес:** `::/128` (всички битове са 0).
- **Loopback адрес:** `::1/128` (еквивалент на 127.0.0.1).
- **Multicast:** `FF00::/8` (започва с префикс 11111111 в двоичен вид).
- **Link-Local Unicast:** `FE80::/10` – адреси за комуникация само в рамките на локалния физически сегмент.
- **Global Unicast:** Всички останали адреси – за стандартна глобална комуникация в Интернет.

4 TCP – процедура на трикратно договаряне

TCP (Transmission Control Protocol) е надежден, ориентиран към връзка (*connection-oriented*) протокол от транспортния слой. Неговата цел е да осигури безгрешен двупосочен пренос на байтов поток от край-до-край, справяйки се успешно със загубени, дублирани или разбъркани пакети чрез механизма на поредни номера (*sequence numbers*) в хедъра на сегментите. Той включва управление на потока (*flow control*) чрез полето *window size*.

Преди да започне какъвто и да е обмен на данни между клиента и сървъра, трябва задължително да се установи виртуална връзка чрез процедурата **трикратно договаряне (Three-Way Handshake)**. Тя преминава през три последователни стъпки:

1. **SYN (Synchronize)**: Клиентът изпраща първоначален сегмент към сървъра, с който заявява желание за установяване на връзка. В този сегмент е вдигнат флагът **SYN**. Клиентът генерира и изпраща произволен начален пореден номер (*Initial Sequence Number - ISN*), например $Seq = x$, с който ще номерира своите данни.
2. **SYN + ACK (Synchronize-Acknowledge)**: Сървърът получава сегмента и ако е готов да приеме връзката, отговаря със сегмент, в който са вдигнати едновременно флагите **SYN** и **ACK**.
 - Чрез флага **ACK** сървърът потвърждава получаването на клиентския SYN, като записва номер за потвърждение $Ack = x + 1$ (очаква следващия байт $x + 1$).
 - Чрез флага **SYN** сървърът указва своя собствен начален пореден номер за данните, които той ще предава в обратната посока, например $Seq = y$.
3. **ACK (Acknowledge)**: Клиентът получава отговора от сървъра и изпраща финален потвърдителен сегмент с вдигнат флаг **ACK**. Номерът за потвърждение се установява на $Ack = y + 1$ (с което се потвърждава получаването на SYN от сървъра), а поредният номер е $Seq = x + 1$.

След успешното завършване на тази стъпка се установява *full-duplex* връзка и двете страни могат да започнат реален и надежден обмен на приложни данни.

5 Хипертекстов протокол HTTP

HTTP (Hypertext Transfer Protocol) е приложен протокол, работещ на принципа клиент-сървър (заявка-отговор / *request-response*). Обикновено клиентите са уеб браузъри, а сървърите са уеб сървъри, предоставящи достъп до обекти, наречени **ресурси**.

5.1 Основни принципи и транспорт

HTTP представлява чист текстов протокол. HTTP е **stateless** протокол (без състояние), което означава, че всяка заявка се разглежда сама за себе си и сървърът не пази информация за минали състояния и потребителски контекст между отделните заявки. За пренос на състояние на по-високо ниво се използват бисквитки (*cookies*).

HTTP е концептуално независим от транспортния слой и може да работи върху всеки протокол, гарантиращ надежден транспорт на данни. В исторически план и при по-ранните си версии протоколът работи стандартно върху **TCP** сокети на порт **80** (или порт 443 за криптирания му вариант HTTPS). Съвременните версии на HTTP поддържат постоянни връзки (*persistent connections*), което позволява един TCP сокет да се преизползва за няколко последователни заявки, но това е оптимизация и не нарушава stateless характера му.

В зависимост от конкретната версия обаче, технологичният стек за транспорт се променя:

- **HTTP/1.1** използва базово **TCP**.
- **HTTP/2** също използва **TCP** с добавено мултиплексиране.
- Съвременният **HTTP/3** използва изцяло протокола **QUIC** върху **UDP** вместо TCP, с цел намаляване на закъснението при установяване на връзка и избягване на блокирането на линията (head-of-line blocking).

5.2 Структура на съобщенията и методи

Всяко съобщение (заявка или отговор) съдържа:

- **Стартов ред** (указващ метода и Request URI при заявка, или статус код при отговор). Ресурсът се идентифицира чрез **URI** (Uniform Resource Identifier), който може да бъде определен по местоположение (**URL**) или по име (**URN**).
- **Хедъри (Headers)** – съдържат метаданни за съобщението.
- **Тяло (Response/Request Body)** – съдържа самото съдържание (HTML, изображения, JSON данни) и е отделено от хедърите с празен ред.

Протоколът дефинира методи за указване на семантиката на действието над ресурса. Основните методи са:

- **GET**: Изисква репрезентация на конкретен ресурс. Трябва да се използва единствено и само за безопасно извличане на данни, без да променя състоянието им.
- **HEAD**: Напълно идентичен с **GET**, но изисква сървърът да върне само стартовия ред и хедърите (*response header*), без да изпраща самото тяло (*response body*). Използва се за бърза проверка на ресурса.

- **POST:** Изпраща данни (в *request body*) към сървъра, които водят до промяна на съществуващ ресурс или създаване на нов подресурс.
- **PUT:** Заменя изцяло текущото представяне на указания ресурс с данните, изпратени в тялото на заявката (*request body*).
- **PATCH:** Използва се за извършване на частична модификация над указания ресурс, базирана на изпратеното тяло.
- **DELETE:** Изтрива изрично указания в Request URI ресурс.
- **CONNECT:** Отваря мрежов тунел към сървъра, идентифициран от ресурса (често се използва при прокси сървъри).
- **OPTIONS:** Използва се за извличане и проверка на поддържаните HTTP методи от сървъра за конкретния URL.
- **TRACE:** Извършва loopback тест по пътя на заявката до ресурс, като кара сървъра да върне обратно точно това съобщение, което е получил, за диагностични цели.

Заклучение

Компютърните мрежи се изграждат върху йерархични архитектури от протоколи и услуги. OSI моделът предоставя теоретична рамка за разделяне на комуникационния процес на слоеве, докато TCP/IP представлява реално използваната архитектура в Интернет. Протоколите IPv4 и IPv6 осигуряват логическа адресация и маршрутизация, като IPv6 решава проблема с ограниченото адресно пространство на IPv4 чрез 128-битово адресиране и опростен хедър. TCP гарантира надежден транспорт чрез механизма на трикратното договаряне, а HTTP е основният приложен протокол за обмен на ресурси в Световната мрежа.